Lesson Notes

MODULE 1 | LESSON 2

VALUE AT RISK (VaR)

Reading Time	120 minutes
Prior Knowledge	Ability to read in from datasets from files or via data feeds using software, some understanding of statistical distributions, notion of simulations, some coding ability to make concepts operational.
Keywords	Value at Risk (VaR) and Expected Shortfall (ES, or cVaR), methods to calculate VaR, Monte Carlo simulations, VaR with non-stationary distributions, main criticisms of VaR, going from 1-day VaR to n-day VAR

In the previous lesson, we looked at important statistics about the return of portfolios and learned how to assess performance. In this lesson, we will examine how to measure the riskiness of portfolios or investments. This is not as simple as it might look because one has to deal with problems stemming from the unknown distribution of returns or, worse, the fact that the distribution may be non-stationary as financial markets experience changes of regime and/or volatility tends to present itself in clustered form.

1. The Role of Value-at-Risk (VaR)

Portfolio performance does not address one key question: how risky is an investment, or more bluntly, how much could we lose?

One could say that in the worst case scenario we could lose all or 100% of our investment (through either financing or derivatives), while in the best case scenario, we could make a boatload of money. However, this answer is not very useful in practice neither to provide a measure or risk nor to manage it. Instead, it would seem that being able to state something like "our portfolio would not lose more than, say 10% with a probability of 95%" has much more value for a practitioner. This is what Value at Risk (**VaR**) is supposed to accomplish.

So, what is Value at Risk? It refers to the maximum potential loss which shall not be exceeded during a specified period of time with a given probability level.

Our goal in this lesson is to learn as much as possible about several of the nuances that are involved using VaR. In order to make the definition above more operational we need to translate the concept to a more operational format. VaR requires two parameters:

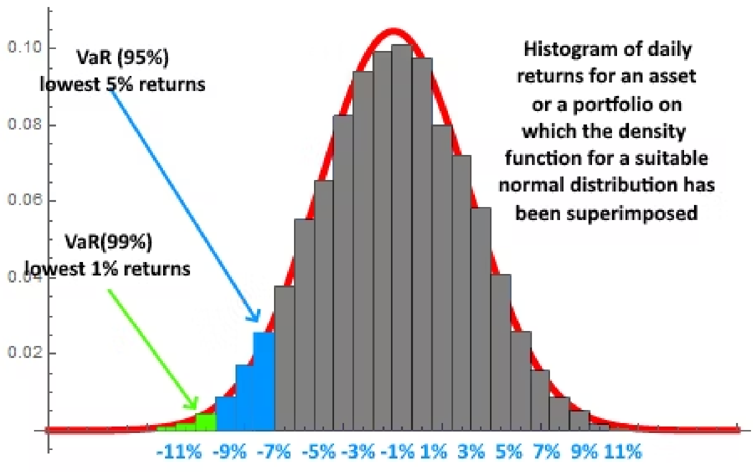
- a confidence level $(1-\alpha)$; and
- a time horizon or reference period (T).

Furthermore, we will use $\mathcal{L}$ to denote the (daily) losses over the holding period T and we will use it as $\mathcal{L}=-\mathcal{R}$, with $\mathcal{R}$ representing the (daily) returns on our investment or portfolio. Sometimes, it is easier to communicate in terms of losses instead of returns turned negative. For a portfolio containing short positions, losses would refer to the securities having positive returns.

We use the symbol $\mathrm{VaR}(1-\alpha)$ to denote the $(1-\alpha)$-th percentile, or the α-th upper percentile of $\mathcal{L}$, that is the lower α-th percentile for returns. Therefore, for continuous loss distributions, we can formally define $\mathrm{VaR}(\alpha)$ as the number such that:

$$Pr[\mathcal{L} \geq \text{VaR}(1-\alpha)] = \alpha = Pr[\mathcal{R} \leq -\text{VaR}(1-\alpha)]$$
$$Pr[\mathcal{L} \leq \text{VaR}(1-\alpha)] = 1-\alpha = \Pr[\mathcal{R} \geq -\text{VaR}(1-\alpha)].$$

Figure 1: The Concept of VaR via a picture



If we stated that our portfolio could lose $\$10$ millions over a time horizon of one month with confidence level 98% , this would be equivalent to saying that the 1-month $\text{VaR}(1-\alpha=0.98)$ is $10,000,0000$, or that there is a probability of 98% not to lose more than $ 10 millions over the next month. Equivalently, this also tells us that there is a probability of 2 % to lose more than $ 10 millions over the next month.

When the joint distribution of returns for several stocks are normally distributed (or more generally speaking, have a joint distribution that belongs to the class of elliptically contoured distributions), because of diversification, the VaR for a portfolio is less than or equal to the sum of the individual VaRs for each of the individual assets in the portfolio. This happens because in this case $\text{VaR}(a-\alpha)$ is determined as $\text{VaR}(1-\alpha) = Mq(\alpha)$ where M is the asset under management and $q(\alpha)$ is the suitable percentile for the elliptically contoured distribution. This family included the normal distributions and the t-distributions among many others. We can show this with two assets, A and B and assuming that both assets have zero expected returns. Also, we have M_A invested in asset A, M_B invested in asset B, with $M = M_A + M_B$ the value of our portfolio. Then, if $w_A = M_A/M$ and $w_B = M_B/M$,

$$\begin{aligned}\text{VaR}_p^2 &= M^2 q(\alpha)^2 (w_A^2\sigma_A^2 + w_B^2\sigma_B^2 + 2w_A w_B \rho_{AB}\sigma_A\sigma_B) \\ &= (q(\alpha)M_A\sigma_A)^2 + (q(\alpha)M_B\sigma_B)^2 + 2\rho_{AB}(q(\alpha)M_A\sigma_A)(q(\alpha)M_B\sigma_B) \\ &= \text{VaR}_A^2 + \text{VaR}_B^2 + 2\rho_{AB}\text{VaR}_A \cdot \text{VaR}_B \leq (\text{VaR}_A + \text{VaR}_B)^2.\end{aligned}$$

Taking square roots on both sides yields the desired conclusion. In general, for m asset portfolios:

$$\text{VaR}_p \leq \sum_{i=1}^{m} \text{VaR}_i,$$

with equality holding only when the correlations among all assets are equal to 1. In Section 5 we provide one example where VaR fails to be subadditive in general.

It is also possible to express VaR as a relative loss (% loss) and not just as an amount. Some books or papers distinguish these two ways to measure VaR using $\text{VaR}(\alpha)$ for amount of loss and $\overline{\text{VaR}}(\alpha)$ when the loss is measured as a % loss.

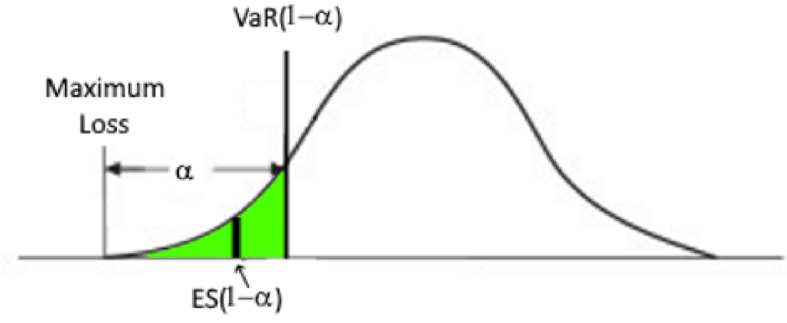
VaR is not completely satisfactory because it primarily tells us that we have a, say 2% probability of losing more than 10 % over a one-month period. However, a portfolio manager is likely concerned about the expected loss whenever the loss exceeds the VaR threshold. To address this concern, we introduce the expected loss given that the loss exceeds the VaR threshold. This measure has been given the technical name of **expected shortfall** (**ES**) (also known as **conditional value at risk** (**cVaR**)) and it is formally defined as:

$$\text{ES}(1-\alpha) = \frac{\int_0^\alpha \text{VaR}(1-s)\,ds}{\alpha}.$$

or, using the language of conditional expectations (and assuming for simplicity that losses have a continuous distribution function),

$$\text{ES}(1-\alpha) = E[\mathcal{L}|\mathcal{L} \geq \text{VaR}(1-\alpha)]$$

Figure 2: The Concept of Expected Shortfall or Conditional VaR



How does one choose a confidence level? Usually, the choice depends on the specific use of VaR. A financial institution monitoring the daily activities of its traders is likely to use a 95% confidence level. However, when dealing with capital requirements, supervisory authorities usually adopt a 99% confidence level. On the other hand, the Basel Committee on Banking Supervision (BCBS) sets the confidence level at a remarkably high 99.9%.

2. Methods to Calculate VaR

There are two main methodologies for VaR calculation:

- **parametric method**, which uses a specific parametric distribution function like a normal distribution or a t-distribution; and
- **non-parametric or historical method**, which uses the empirical distribution obtained directly from the historical data.

It is possible to combine the two methods in what is called a **semi-parametric method**.

Some may also list Monte Carlo simulations as a separate group, but technically, that is just a particular way to handle parametric distributions or even non-parametric ones when simulations are implemented via bootstrapping techniques.

As we are aware, economic and financial time series are usually not stationary. While assuming stationarity may not always be wrong as there are periods in which underlying risk factors are stable, over longer time horizons, that assumption does not hold. As such, we distinguish between two additional types of calculations:

- **unconditional VaR,** based on the unconditional distribution of the underlying risk factors; and
- **conditional VaR**, which assumes that the relevant distribution and/or parameters could change over time.

The non-parametric VaR estimation is simple in principle. Suppose that we want a confidence coefficient of $1-\alpha$ Then, all that is required is to estimate the α upper quantile of the loss distribution for the sample of historic returns. We will call such quantile $q(\alpha)$ and its sample equivalent $\hat{q}(\alpha)$.

If our current portfolio is worth M then the nonparametric estimate of VaR is:

$$\widehat{\text{VaR}}_{np}(1-\alpha) = M \cdot \hat{q}(\alpha).$$

The expected shortfall has a more complicated expression but in reality is a simple concept, i.e.:

$$\widehat{\text{ES}}_{np}(1-\alpha) = M \cdot \frac{\sum_{i=1}^{n} r_i \cdot I(r_i < \hat{q}(\alpha))}{\sum_{i=1}^{n} I(r_i < \hat{q}(\alpha))}.$$

The indicator function $I(r_i < \hat{q}(\alpha)) = 1$ if the r_i-th return is less than $\hat{q}(\alpha)$ and 0 otherwise, $\sum_{i=1}^{n} I(r_i < \hat{q}(\alpha))$ counts the number of returns below the threshold $\hat{q}(\alpha)$.

Hence, the formula supplied is just a fancy (but mathematically rigorous) way to write the sample mean of returns that fall below the $\widehat{\mathrm{VaR}}_{np}(1-\alpha)$ threshold.

Example. *This is a long example that will accompany us through several steps and that mixes Python code and results. From time to time, we will interrupt the example to develop some theory and then return back.*

We work with the daily returns for AMZN stock during the period from Jan 1, 2015 to Dec 31, 2021. If we wanted to calculate the $\mathrm{VaR}_np(\alpha)$ for a1-day period for $\alpha = 0.10, 0.05$, and 0.01 we could use the code below. The small program also produces a histogram of the daily returns and superimposes the suitable normal density. We notice that the histogram exhibits heavier tails than those of the normal distribution. Because of this, the nonparametric VaR could be actually more desirable in this instance. For the sake of simplicity, we assume $M =$ \$1.

```
# import required libraries:

import pandas as pd
import numpy as np
from pandas_datareader import data as pdr
import yfinance as yf
yf.pdr_override()
import matplotlib.pyplot as plt
import math
import seaborn
import datetime
from datetime import date

from scipy.stats import norm
from tabulate import tabulate

# read the data for the required stock
ticker ="AMZN"
start_date = datetime.date(2015,1,1)
end_date = datetime.date(2021,12,31)

data = pdr.get_data_yahoo(ticker, start=start_date, end=end_date)['Adj Close']
stock_returns = data.pct_change()
stock_returns= stock_returns.dropna()

VaR_90=np.quantile(stock_returns, .1)
VaR_95=np.quantile(stock_returns, .05)
VaR_99=np.quantile(stock_returns, .01)

table_VaR = [['90%', VaR_90], ['95%', VaR_95], ['99%', VaR_99]]
print(tabulate(table_VaR, headers=['Confidence level', 'VaR']))

ES_90 = stock_returns[stock_returns < VaR_90].mean()
ES_95 = stock_returns[stock_returns < VaR_95].mean()
ES_99 = stock_returns[stock_returns < VaR_99].mean()
```

```
table_ES = [['90%', ES_90], ['95%', ES_95], ['99%', ES_99]]
print(tabulate(table_ES, headers=['Confidence level', 'ES']))
```

The code above produces the following output:

```
Confidence level          VaR
------------------  ----------
90%                 -0.0181798
95%                 -0.0279091
99%                 -0.0523702
```

and

```
Confidence level           ES
------------------  ----------
90%                 -0.0319155
95%                 -0.0415431
99%                 -0.0634476
```

We compare now the histogram of the stock's returns over the period with the estimated parametric normal family. The following lines are also taken from the previous program:

```
#pip install pandas-datareader  #Make sure you have at least version 0.10
from scipy.stats import t
mean = np.mean(stock_returns)
stdev = np.std(stock_returns)
nu, mu, sigma = t.fit(stock_returns)
stock_returns.hist(bins=40,density=True,
                        histtype='stepfilled', alpha=0.5)
x=np.linspace(mean-4*stdev, mean + 4*stdev, 100)
plt.plot(x,norm.pdf(x, mean, stdev), "r", label="normal")
plt.plot(x,t.pdf(x, nu,mu, sigma), "g", label="t-dist")
plt.xlabel('returns')
plt.ylabel('frequency')
plt.legend(loc="upper right")
plt.show
```

Figure 3: Comparing the Histogram of Amazon's Daily Returns with the Estimated Normal and t-Distribution

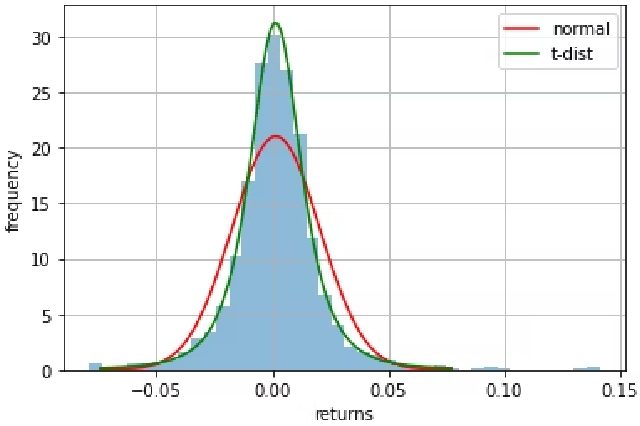


Figure 3 shows that the tails of the histogram of daily returns of AMZN during the reference period are heavier than those for the normal distribution. In this case, using a parametric VaR with a normal distribution could underestimate the Value at Risk, and the t-distribution does a better job at capturing the histogram. So our next step will be to incorporate some of this in our approach to estimating VaR.

Non-parametric estimates using sample quantiles, like in our example, have the advantage that practitioners are not required to make assumptions about the distribution of returns. However, they do work better when the sample size is large and α is not too small. In particular, if α is chosen small, then a very large sample size is required to obtain a reliable estimate of the relevant quantile. The reason is that we have very few data *observations* on the tails compared to the central part of the distribution, so the statistical estimates can lack precision.

When we assume that the stock returns are modeled by a specific parametric distribution, $F(r, \theta = (\theta_1, \ldots, \theta_p))$, all one has to do is obtain an estimate of the parameter(s) $\theta_1, \ldots, \theta_p$ using either MLE (maximum likelihood) or MOM (method of moments) applied to the historical returns available. Then, in general, using a parametric technique, we have

$$\widehat{\text{VaR}}_p(1-\alpha) = M \cdot F^{-1}(\alpha, \hat{\theta}),$$

where M is again the (dollar) value of the asset owned.

Example (continued). Let's say that we are willing to assume that the underlying distribution for the historical returns of AMZN is normal, i.e.:

$$F(r, \theta) = \Phi(\mu, \sigma^2)$$

where $\Phi(\cdot)$ is the cumulative distribution function for a normal family and μ and σ^2 are the two parameters required to describe it.

For a normal distribution, the MLE estimators are $\hat{\mu} = \bar{x}$, the sample mean and $\hat{\sigma}^2 = s^2$, the sample variance (divided by n instead of $n-1$, but for large samples, that hardly makes any difference). Thus:

```
mean
Out[4]: 0.0015299261202024064
stdev
Out[5]: 0.018958006671941956
pow(stdev, 2)
Out[9]: 0.0003594060169733957
```

i.e., we have found $\hat{\mu} = 0.00153$ and $\hat{\sigma} = 0.01896$.

This means that the parametric model for returns is $r \sim N(\mu, \sigma^2)$, which means also that

$$\frac{r-\mu}{\sigma} \sim N(0,1),$$

and if $q(\alpha)$ represents the α -th lower percentile of a standard normal distribution, simple elementary properties of the normal random variables yield:

$$\widehat{\mathrm{VaR}}_p(1-\alpha) = F^{-1}(\alpha, \hat{\theta}) = \hat{\mu} + \hat{\sigma} \cdot q(\alpha).$$

Then, a few lines of Python code yield:

```
import scipy.stats as st
q_010 = st.norm.ppf(.10)
q_005 = st.norm.ppf(.05)
q_001 = st.norm.ppf(.01)

VaR_90 = mean + stdev*q_010
VaR_95 = mean + stdev*q_005
VaR_99 = mean + stdev*q_001

table_VaR = [['90%', VaR_90], ['95%', VaR_95], ['99%', VaR_99]]
print(tabulate(table_VaR,
headers=['Confidence level (normal)', 'VaR']))

Confidence (normal)          VaR
---------------------  ----------
90%                    -0.0227657
95%                    -0.0296532
99%                    -0.042573
```

As we suspected from looking at the figure with the histogram, the $\mathrm{Var}_p(\alpha)$ is lower (bigger losses) than $\mathrm{Var}_{np}(\alpha)$ for $\alpha = 0.10$ and $\alpha = 0.05$. The opposite is true for $\alpha = .01$.

The computation of $\mathrm{ES}(1-\alpha)$ is less straightforward but reasonably simple. If we use a normal distribution, we get

$$\widehat{\mathrm{ES}}_p(1-\alpha) = M \cdot \left(\hat{\mu} - \hat{\sigma}\left(\frac{\phi(\Phi^{-1}(\alpha))}{\alpha}\right)\right),$$

where $\hat{\mu} = \bar{x}, \hat{\sigma} = s$ and $\phi(\cdot), \Phi(\cdot)$ are the pdf and the cdf of a standard normal distribution, respectively.

Here are the few lines of Python code (to be used together with the rest of the code, of course):

```
ES_90 = +mean - stdev*(norm.pdf(q_010)/.10)
ES_95 = +mean - stdev*(norm.pdf(q_005)/.05)
ES_99 = +mean - stdev*(norm.pdf(q_001)/.01)

table_ES = [['90%', ES_90], ['95%', ES_95], ['99%', ES_99]]
print(tabulate(table_ES,
headers=['Confidence level (normal)', 'ES']))

Confidence level (normal)          ES
---------------------------  ----------
90%                          -0.0317411
```

```
95%                         -0.037575
99%                         -0.0489972
```

Following our discussion about the tails of probability distributions and having noticed that the histogram seems to have heavier tails than the normal, we could compute a new parametric VaR using a t-distribution instead of a normal. Say that the chosen parametric family of distribution is a$t(\nu, \mu, \sigma)$, where ν represents the degrees of freedom, μ is the location parameter and σ the scale parameter.

This distribution can be considered the outcome of starting with $X \sim t(\nu)$, a standard t distribution with ν degrees of freedom and considering the new random variable

$$Y = \mu + \sigma X.$$

In this case:

$$\widehat{\mathrm{VaR}}_p(1-\alpha) = M \cdot \left(\hat{\mu} + \hat{\sigma} \cdot F_{\hat{\nu}}^{-1}(\alpha)\right)$$

and

$$\widehat{ES}_p(1-\alpha) = M \cdot \left(\hat{\mu} - \hat{\sigma}\left(\frac{f_{\hat{\nu}}(F_{\hat{\nu}}^{1-}(\alpha))}{\alpha}\right) \cdot \left[\frac{\hat{\nu} + (F_{\hat{\nu}}^{-1}(\alpha))^2}{\hat{\nu} - 1}\right]\right)$$

where $f_\nu(\cdot)$ and $F_\nu(\cdot)$ stand for the pdf and cdf of a standard t-distribution with ν degrees of freedom, respectively and $\hat{\mu}, \hat{\sigma}$, and $\hat{\nu}$ are the estimates (usually MLEs) of the unknown parameters.

To estimate the parameters in Python, we can use the **t.fit** command. However, to run this module, the dataset must be of type array. **t.fit**, by default, uses ML to estimate the parameters, but MOM is also available. Nevertheless, one has to be careful that the t-distribution does not have certain moments for all values of degrees of freedom.

Example (continued). Returning to our case,

```
from scipy.stats import t
from array import array

stock_ret_array = stock_returns.to_numpy()
nu, mu, sigma = t.fit(stock_ret_array)
##############
t.fit(stock_ret_array)
Out[238]: (2.9432547431402076, 0.0014122907770148384, 0.011709608784175964)
```

We found that $\hat{\nu} = 2.94345, \hat{\mu} = 0.001412$, and $\hat{\sigma} = 0.01171$, and these estimates can be used to estimate the required VaR and ES.

```
q_010 = st.t.ppf(.10, nu, mu, sigma)
q_005 = st.t.ppf(.05, nu, mu, sigma)
q_001 = st.t.ppf(.01, nu, mu, sigma)

VaR_90 = q_010
VaR_95 = q_005
VaR_99 = q_001

table_VaR = [['90%', VaR_90], ['95%', VaR_95], ['99%', VaR_99]]
print(tabulate(table_VaR,
```

```
headers=['Confidence level(t-dist.)', 'VaR']))

Confidence level with (t-dist.)          VaR
---------------------------------  ----------
90%                                -0.0178653
95%                                -0.0263626
99%                                -0.0525860
```

3. Monte Carlo Simulations

In principle, this method is very simple and can be seen as an extension of the parametric approach to VaR. In essence, in its most basic form of implementation, we start with a parametric model and simulate, say $n = 1,000,000$ random variables or patterns of prices, over a desired period of time. Then, we can compute the percentile for the required level of confidence and we are done.

Like in any parametric model, the distribution is specified, but the parameters of the same must be estimated using historical data for instance.

Using again the example we were working on previously, here are a few basic lines of code to implement the concept:

Example (continued).

```
n_sims = 1000000
sim_returns = np.random.normal(mean, stdev, n_sims)
sim_VaR = np.percentile(sim_returns, 1)
print('1-day simulated VaR is ', sim_VaR)

1-day simulated 99% VaR is  -0.04257348749817126
```

If a practitioner needed, say a 1-month VaR instead of a 1-day VaR for any level of confidence, all they must do is adjust the parameters to reflect *average return and volatility over a period of 250/12 = 21 days* , but otherwise, the same few lines suffice. However, one needs to be aware that for a 1-month VaR, estimates would be based on fewer observations and thus less accurate.

Of course, there is minimal advantage in using Monte Carlo simulations in such a simplistic way vs. using the parametric or non-parametric methods. Also, if we lack reliable data, the parameters could not be estimated with sufficient precision.

As one has probably guessed already, this is not the end of the story. Monte Carlo methods are very powerful in allowing us to introduce possible scenarios. For instance, let's say that we are interested in a 30-day VaR, but we sense that certain events could take place that could affect the prices and therefore returns of a stock or some portfolio. When using simulations, it is reasonably easy to introduce jumps or model individual risks—something that could not be done using the other two approaches.

So far, we considered the case in which we have one stock. That could very well have been a portfolio considered as an aggregate. Where the approach shows its strength is in handling complex situations involving many assets at the same time because we could simulate each asset separately and consider specific sources of risk for each of the components. However, while in principle it is possible to simulate the returns of each of the, say m assets in the portfolio and then aggregate the returns via a linear combination with known weights, that would not be the most general way to do it.

Assets are not completely uncorrelated with each other and some investment classes may even exhibit strong dependence. This dependence should be taken into consideration. During the Great Financial Crisis, for instance, the world became painfully aware that investments were way more correlated than previously assumed. Monte Carlo simulations can help us model these situations. We will use a multivariate normal approach, but one could use a multivariate t-distribution or other types of distributions. Nevertheless, some are easier than others to use and perhaps easier to understand as a teaching tool as well.

Assume that we have m assets in our portfolio and that these are linearly priced or approximately so.

What do we mean by this? Clearly, if one of the assets is a stock, then its value is a linear function of its price. But, the relationship between a bond and the interest rate is not linear, nor is the relationship between a call option and the price of its underlying stock. However, here we assume either that the relationship is linear or it can be approximated reasonably well by a linear relationship.

In the literature, this is sometimes called a linear portfolio. The situation can be described as follows:

- S is an $m \times 1$ vector of market prices and interest rates;
- Δt represents the time horizon over which we would like to perform risk measurements;
- $V(S,t)$ stands for the value of our portfolio at time t and market prices S;
- $L = V(S,t) - V(S + \Delta S, t + \Delta t)$ is the loss in the value of our portfolio over the time frame Δt ;
- when we say that we have a linear portfolio, we mean that $\Delta V = \beta^T \Delta S$, where β is a vector of sensitivities.

If one of the assets, say W is a derivative, we know from previous courses that

$$\Delta W \approx \frac{\partial W}{\partial t}(\Delta t) + \beta(\Delta S) + \frac{1}{2}\Gamma(\Delta S)^2$$

with $\beta = \frac{\partial W}{\partial S}$, and $\Gamma = \frac{\partial^2 W}{\partial S^2}$, and $\Delta S, \Delta T$ just the discrete time change in value of S and t, respectively. If the first and third terms in the above formula are small, we could approximate the asset in a linear fashion. Incidentally, this tells us that estimating the sensitivities can require some work as well.

To illustrate how the methods work, assume that the relative changes in risk factors (think of returns in case the risk factors are the stock prices) for the next day have a multivariate normal distribution:

$$r_{T+1} \sim N(\mu, \Sigma)$$

where T is the current time, μ an $m \times 1$ vector and $\mathbf{\Sigma}$ is an $m \times m$ variance-covariance matrix, and both need to be estimated. Now, using the assumption of linearity, we have in general that

$$\frac{V_t}{V_{t-1}} = \beta^T r_t$$

which is why we insisted on a linearly priced portfolio. Then, this means that

$$\frac{V_{T+1}}{V_T} \sim N(\beta^T \mu, \beta^T \mathbf{\Sigma} \beta).$$

Now, one gets

$$\widehat{\text{VaR}}(\alpha) = \beta^T \hat{\mu} + \Phi^{-1}(\alpha)\sqrt{\beta^T \widehat{\Sigma} \beta},$$

with $\Phi^{-1}(\alpha)$, as usual, the α quantile for the standard normal distribution.

The method just described is also known as the **variance-covariance method** and is very suitable for simulations as well. This method allows for including correlations among risk factors in the calculations. We stress the fact that this approach can be difficult to implement if we remove that assumption of linear pricing, but it would be still doable, albeit more complex.

In this day and age, simulating a multivariate normal in Python or most other packages is straightforward:

```
import numpy as np
# specify mean and variance-covariance matrix
mu = [1, 2]
```

```
Sigma = [[5, 1], [1, 4]]
n_sim = 100000
sim_mult_normal = np.random.multivariate_normal(mu, Sigma, n_sim)
```

Simulating from other multivariate distributions can be more challenging. For the multivariate t-distribution in Python, one implementation is presented here: multivariate-t-distribution.

The R programming language has more built-in packages for this purpose, and if you are dealing with tasks related to VaR and ES, keep this in mind. Regardless, the multivariate t-distribution is quite important because it provides a good model for the returns of individual assets as seen in our earlier example. In addition, like the normal, the multivariate t-distribution has the property that the univariate marginal distributions have univariate t-distributions.

We can create a vector $Y \sim t(\nu, \mu, \Omega)$ by letting

$$Y = \mu + \sqrt{\frac{\nu}{V}} \cdot X \quad \text{where} \quad X \sim N(0, \Omega), V \sim \chi^2(\nu)$$

and V, X are independent.

Practitioners must note that Ω is the variance-covariance matrix for X, but not for Y The variance-covariance matrix for Y is

$$\text{var}(Y) = \frac{\nu}{\nu - 2}\Omega$$

and, as one can see, it is not even defined for $\nu <= 2$. However, Ω and Σ are just multiples of each other. If all the terms off the main diagonal of Ω are zero, then we would obtain vectors $m \times 1$ of independent univariate components.

In applications, there is still a need to estimate ν, μ, and Ω. This is usually done using MLE.

If we let Y describe the m individual assets in the portfolio and, for simplicity, we assume these are all stocks, then the linear pricing assumption is satisfied. Furthermore, in this case, the sensitivity factors β are all equal to 1.

In this scenario, if we start with $Y \sim t(\nu, \mu, \Omega)$ and calculate the estimates $\hat{\nu}$, $\hat{\mu}$, and $\hat{\Omega}$ then the portfolio, $\Pi = w^T Y$ has a univariate t-distribution with mean $w^T \hat{\nu}$ and variance $\frac{\hat{\nu}}{\hat{\nu}-2} w^T \hat{\Omega} w$ where w is the $m \times 1$ vector of the weights of each of the m-assets in the portfolio.

And here is some Python code to do just that:

```
# simulating from multivariate t distribution.
import numpy as np
from scipy.stats import chi

def multivariate_t_rvs(m, S, df=1, n=1):
m = np.asarray(m)  # procedure requires a type array
d = len(m)     # distribution is d-dimensional
v = np.random.chisquare(df, n)
z = np.random.multivariate_normal(np.zeros(d),S,(n,))
return m + z*np.sqrt(df/v)[:,None]

# using the function:
n=5  # number of simulations
df=3 # degrees of freedom
mu = [1, 2, -1]
Omega = [[1, .5, -0.3], [.5, 2, 0], [-0.3, 0, 2]]
```

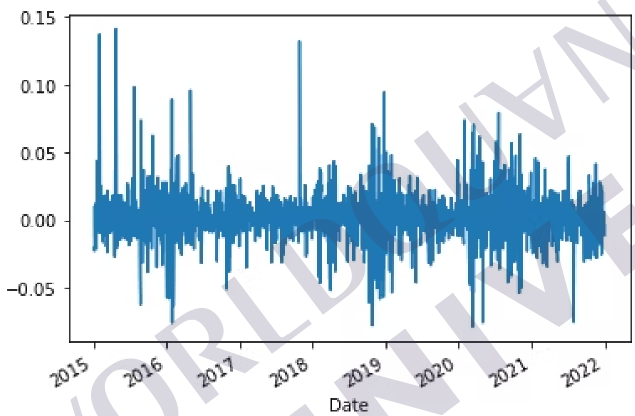
```
simulated_values = multivariate_t_rvs(mu, Omega, df, n)
#######  results:
print(simulated_values)
[[-0.15946801 -1.37601537 -8.26035747]
[ 2.2442207   0.27457816 -2.11121623]
[ 4.54699115  3.48056433 -3.79188743]
[ 2.80460098  3.57288629 -4.10279591]
[ 1.96680752  2.07122964 -2.89336293]]
```

This allows us to devise simulations where stock returns exhibit heavy tails, can be correlated among themselves, and will produce more accurate values of VaR and ES for our portfolios or investments.

4. VaR with Non-Stationary Distributions

In what we did so far, we made a very strong assumption: The distributions are stationary, meaning they do not change over time. However, we know that this is not the case in real life, not always at least since real data tend to exhibit a special type of heteroscedasticity known as volatility clustering.

Figure 4: Example of volatility clusters over different sub periods using returns on Amazon stock.



Therefore, one has to account for the time-varying volatility. Assume that we have n returns $r_1, r_2, \ldots, r_n$ and we need to estimate VaR and ES for the next return r_{n+1}. Let $\hat{\mu}_{n+1|r_1,\ldots,r_n}$ and $\hat{\sigma}_{n+1|r_1,\ldots,r_n}$ be the estimated and conditional variance of r_{n+1}, which are obtained conditionally on the current information available to us, i.e.$\{r_1, r_2, \ldots, r_n\}$. If we use a normal distribution, we could just compute VaR and ES using the same formula as before:

$$\mathrm{VaR}(1-\alpha) = \hat{\mu}_{n+1|r_1,\ldots,r_n} + \hat{\sigma}_{n+1|r_1,\ldots,r_n}\Phi^{-1}(\alpha)$$

and

$$\mathrm{ES}(1-\alpha) = \hat{\mu}_{n+1|r_1,\ldots,r_n} + \hat{\sigma}_{n+1|r_1,\ldots,r_n}\left(\frac{\phi(\Phi^{-1}(\alpha))}{\alpha}\right).$$

A similar results holds for the t-distribution if we

$$\widehat{\mathrm{VaR}}_p(1-\alpha) = M \cdot \left(\hat{\mu}_{n+1|r_1,\ldots,r_n} + \hat{\sigma}_{n+1|r_1,\ldots,r_n} \cdot F^{-1}_{\hat{\nu}_{n+1|r_1,\ldots,r_n}}(\alpha)\right)$$

and

$$\widehat{ES}_p(1-\alpha) = M \cdot \left(\hat{\mu}_{n+1|r_1,\ldots,r_n} - \hat{\sigma}_{n+1|r_1,\ldots,r_n} \times \left(\frac{f_{\hat{\nu}_{n+1|r_1,\ldots,r_n}}(F^{1-}_{\hat{\nu}_{n+1|r_1,\ldots,r_n}}(\alpha))}{\alpha}\right)\right) \cdot \left[\frac{\hat{\nu}_{n+1|r_1,\ldots,r_n} + (F^{-1}_{\hat{\nu}_{n+1|r_1,\ldots,r_n}}(\alpha))^2}{\hat{\nu}_{n+1|r_1,\ldots,r_n} - 1}\right].$$

Although it may seem intimidating, in reality, the application is relatively straightforward.

How do we put this into practice? There is more than one way to implement this program. A reasonably simple method is that proposed by Riskmetrics, which uses an **exponentially weighted moving average** (*EWMA*). This method assumes that volatility, measured as σ_t^2 changes over time according to the following process:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)(r_t - \mu)^2$$

with μ the unconditional mean and λ a constant arbitrarily chosen from the interval $(0, 1)$. Riskmetrics suggested $\lambda = 0.94.$ One should note that by the nature of how the exponential distribution decays, exponential weighting states that the older the history, the lower its effect.

If we compared this approach to using historical volatility, since the historical volatility using the most recent n observation is given by the formula, then:

$$\sigma_t^2 = \frac{1}{n}\sum_{i=t-n+1}^{t}(r_t - \mu)^2.$$

Then, simple algebraic manipulations yield

$$\sigma_t^2 = \frac{n-1}{n}\sigma_{t-1}^2 + \frac{1}{n}(r_n - \mu)^2.$$

In particular, we should notice that the historical volatility is a special case of EWMA and is achieved when $\lambda = (n-1)/n$. For n very large, then $(n-1)/n$ approaches 1 and the last observation receives less weight than choosing $\lambda = 0.94$ as proposed by Riskmetrics.

It would be preferable to estimate the value of λ directly from the data, and this leads us to use models like GARCH. In particular, we are going to illustrate this method using a GARCH(1, 1), a typical model for this type of application:

$$r_t = \mu + \epsilon_t,\;\; \epsilon_t \sim N(0, \sigma_t^2)$$
$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$$

This model can be implemented in Python using the following code:

```
from arch import arch_model

model=arch_model(stock_returns, vol='Garch', p=1, o=0, q=1, dist='Normal')
results=model.fit()
print(results.summary())
```

```
                     Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:                  stock   R-squared:                       0.000
Mean Model:             Constant Mean   Adj. R-squared:                  0.000
Vol Model:                      GARCH   Log-Likelihood:                4608.01
Distribution:                  Normal   AIC:                          -9208.02
Method:            Maximum Likelihood   BIC:                          -9186.12
                                        No. Observations:                 1762
Date:                Sun, Jan 09 2022   Df Residuals:                     1761
Time:                        09:29:48   Df Model:                            1
                               Mean Model
============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
----------------------------------------------------------------------------
mu         2.2447e-03  4.750e-04      4.725  2.296e-06 [1.314e-03,3.176e-03]
                            Volatility Model
============================================================================
                 coef    std err          t      P>|t|      95.0% Conf. Int.
----------------------------------------------------------------------------
omega      3.5941e-05  3.846e-06      9.345  9.214e-21 [2.840e-05,4.348e-05]
alpha[1]       0.2000  3.749e-02      5.334  9.583e-08     [  0.127,  0.273]
beta[1]        0.7000  3.561e-02     19.655  5.241e-86     [  0.630,  0.770]
============================================================================

Covariance estimator: robust
```

Our estimated model is thus

$$\widehat{\sigma}_t^2 = 0.0000359 + 0.200\epsilon_{t-1}^2 + 0.700\sigma_{t-1}^2$$

and all estimates are statistically significant at the 95% confidence level. The model can now be used to predict the variance/volatility for our asset:

```
forecasts = results.forecast(horizon=3)
print(forecasts.variance)
h.1       h.2       h.3
Date
...       ...       ...
2021-12-27       NaN       NaN       NaN
2021-12-28       NaN       NaN       NaN
2021-12-29       NaN       NaN       NaN
2021-12-30       NaN       NaN       NaN
2021-12-31  0.000197  0.000213  0.000228
```

The 1-day estimated variance is 0.000197, which corresponds to a daily volatility of $0.0140,$ lower than the one we estimated using the historical values.

We can repeat the estimation using a t-distribution:

$$r_t = \mu + \epsilon_t, \;\; \epsilon_t \sim t_\nu$$
$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$$

```
model=arch_model(100*stock_returns, vol='Garch', p=1, o=0, q=1, dist='t')
```

Note that the multiplication by 100 was done to improve the chances of the estimating process converging to the proper solution and avoiding numerical inaccuracies. When estimating GARCH models, complex numerical procedures are involved and their convergence could be slow or even faulty if numbers get too close to zero and the tolerance is exceeded. Multiplying by 100 is like deciding to use, say, 3% instead of 0.03.

The result is:

```
                    Constant Mean - GARCH Model Results
==============================================================================
Dep. Variable:                    stock   R-squared:                       0.000
Mean Model:               Constant Mean   Adj. R-squared:                  0.000
Vol Model:                        GARCH   Log-Likelihood:               -3331.78
Distribution:   Standardized Student's t  AIC:                           6673.55
Method:              Maximum Likelihood   BIC:                           6700.93
                                          No. Observations:                 1762
Date:                  Sun, Jan 09 2022   Df Residuals:                     1761
Time:                          12:31:36   Df Model:                            1
                               Mean Model
============================================================================
                 coef    std err          t      P>|t|    95.0% Conf. Int.
----------------------------------------------------------------------------
mu             0.1540  3.118e-02      4.939  7.834e-07 [9.290e-02,  0.215]
                            Volatility Model
============================================================================
                 coef    std err          t      P>|t|    95.0% Conf. Int.
----------------------------------------------------------------------------
omega          0.2342  8.356e-02      2.803  5.063e-03 [7.044e-02,  0.398]
alpha[1]       0.1721  4.256e-02      4.044  5.260e-05 [8.869e-02,  0.256]
beta[1]        0.7787  5.081e-02     15.327  5.030e-53   [  0.679,  0.878]
                              Distribution
========================================================================
                 coef    std err          t      P>|t|  95.0% Conf. Int.
------------------------------------------------------------------------
nu             4.0709      0.435      9.365  7.585e-21 [  3.219,  4.923]
```

Note that, again, all estimates are statistically significant at the 95% confidence level and that in this case we have an additional estimate, $\hat{\nu}$, which is the estimate for the degrees of freedom of the t-distribution that is used in our GARCH(1,1) model. To obtain the forecast for the variance, we just need:

```
forecasts = results.forecast(horizon=3)
print(forecasts.variance)

h.1        h.2        h.3
Date
...        ...        ...
2021-12-27       NaN        NaN        NaN
2021-12-28       NaN        NaN        NaN
2021-12-29       NaN        NaN        NaN
2021-12-30       NaN        NaN        NaN
2021-12-31  1.988585  2.125057  2.254822
```


We must recall that we multiplied the stock returns by 100 for the sake of numerical accuracy in running the optimization algorithm, so we need to divide the number by 100^2 (this is the variance!). Hence, the 1-day estimate for the variance is $1.9886 \times 100^{-2} = 0.0001988$ or a volatility of 0.0141. Again, this is smaller than what we found using parametric or non-parametric estimation. It is often the case when using conditional approaches like GARCH.

If we are using a GARCH(1,1) model, we can also compute the unconditional variance using the formula

$$\hat{\sigma}^2_{uncond.} = \frac{\omega}{1-\alpha-\beta} = \frac{.2342}{1-.1721-.7787} = 4.7602$$

which must be divided by 100^2 to yield $\hat{\sigma}^2_{uncond.} = 0.000476$ for the data before the multiplication by 100. That translates to a daily volatility of 0.0218, considerably higher than the conditional one we estimated. The formula also shows that in order for the model to make sense, we need to check the condition that $\alpha + \beta < 1$.

Once the degrees of freedom for the t-distribution are known, we can use the formula we have seen earlier to compute VaR and ES :

$$\widehat{\text{VaR}}_p(1-\alpha) = M \cdot \left(\hat{\mu} + \hat{\sigma} \cdot F_{\hat{\nu}}^{-1}(\alpha)\right)$$

and

$$\widehat{ES}_p(1-\alpha) = M \cdot \left(\hat{\mu} - \hat{\sigma}\left(\frac{f_{\hat{\nu}}(F_{\hat{\nu}}^{1-}(\alpha))}{\alpha}\right) \cdot \left[\frac{\hat{\nu} + (F_{\hat{\nu}}^{-1}(\alpha))^2}{\hat{\nu}-1}\right]\right)$$

We only need to notice that while $\hat{\nu}$ can be taken from the output, $\hat{\lambda}_{n+1|r_1,...,r_n}$ must be computed in the formula for λ and the estimate $\hat{\sigma}_{n+1|r_1,...,r_n}$ that we just found, i.e., $\hat{\sigma}^2_{n+1|r_1,...,r_n} = 0.0001988$.
Hence:

$$\hat{\lambda}_{n+1|r_1,...,r_1} = \sqrt{\frac{\hat{\nu}-2}{\hat{\nu}}} \cdot \hat{\sigma}_{n+1|r_1,...,r_n}.$$

Finally, $\hat{\mu}$ is the estimate "mu" in the output provided, i.e.$\hat{\mu}_{n+1|r_1,...,r_n} = 0.1540/100 = .00154$ (recall we multiplied the returns by 100 to facilitate the convergence of the numerical computations).

One should notice that, in our example, the conditional volatility is below the unconditional, but it is not always so. As the graph shows, we are in a period of less turbulence for the stock, but at different times, we could find that the conditional variance is above the unconditional one. That is also in the spirit of volatility clustering.

This is not the "end of the story" because with time series models there are many other possible models that we could use. For instance, we could use a GARCH(1,2) (it was tried, and it does not produce a better statistical model), a combination of ARMA+GARCH, GARCH-M (GARCH-in-Mean), EGARCH (Exponential GARCH), TARCH (Threshold GARCH), and more. Of course, we could consider multivariate GARCH models as well.

5. Criticism of VaR

Despite the fact that VaR has become the major risk measure in the financial industry worldwide both among industry participants and regulators, VaR has also had its amount of detractors. Among the major shortcomings we find:

Liquidity risk. To understand this weakness, assume as it is often the case that we are working with a daily VaR. Implicit in the calculation of VaR is the assumption that positions can be completely liquidated very quickly, over a period of just a few days at most and at the prevailing market prices. However, during periods of particular economic turbulence, this may not be the case, and when major market participants starts to liquidate their positions, this could also cause a frenzy that induces other investors to do the same (herding behavior, one of the tenets of behavioral finance), and this determines a rapid drop in prices as everyone is trying to sell.

Model risk. We have seen that there are many ways to calculate VaR, but we need to assume some statistical model. The non-parametric approach does not; however, it is not good at estimating very small quantiles unless one has a lot of data. In other words, the non-parametric approach may be suitable for a$95\%\ \text{VaR}$, but not so for a$99\%\text{-VaR}$. Invalid assumptions can affect our calculations. In addition, a model may just look at past factors and fail to incorporate risk factors that are changing over time. This is also a risk that can occur when doing simulations as we are imposing certain distributions and certain relationships among the risk factors.

Quantiles of distributions do not tell us the whole story. As it is conceived, VaR only tells us that we have a probability α to lose more than a certain amount; however, it does not tell us the potential magnitude of those losses. Nevertheless, the expected shortfall allows us to say something about what happens on the negative tails of returns distribution, but it does not prevent institutions from taking very risky positions that could lead to significant losses, despite the low probability (assuming we are using the correct statistical model).

Lack of subadditivity. In an ideal world one expects that, because of diversification, the sum of the risks of two or more individual investments in a portfolio would not exceed the risk of the entire portfolio, i.e., if P_1 and P_2 represent two positions or portfolios, we expect that:

$$\text{VaR}(P_1 + P_2) \leq \text{VaR}(P_1) + \text{VaR}(P_2).$$

However, this is not so with VaR, not always at least.

Example.

Let's assume that we have two assets, A and B. We assume that A has a return of 0% with probability 0.992 and a return of c with probability 0.008. The same is true for asset B. Then, the $\mathrm{Var}(0.99) = 0.$ Now consider the portfolio $A + B$ and assume it will have a return of $2C$ with probability $0.008 \times 0.008 = 0.000064$, a return of c with probability $2 \times 0.008 \times 0.992 = 0.001587$ and a return of 0 with probability $0.98406.$ Then in this case, the 99%-VaR is $(0.99) = c > 0.$

The fact that the VaR of a sum may be higher than the sum of VaRs for each position a portfolio produces can cause financial institutions to create portfolios that tend to be concentrated. This can happen because a financial institution will tend to concentrate on large positions in a few assets, trying to game the VaR. In this scenario, given that the risk centers would be fewer, the VaR would be lower as well. Hence, the failure to properly measure the benefits of diversification is a significant issue.

Modeling individual assets together and also taking into consideration the correlations among the different asset classes does help to alleviate this issue. The following paper contains an interesting discussion on the limitations of diversification: "The Impact of Systemic Risk on the Diversification Benefits of a Risk Portfolio."

6. A Remark: From 1-day VaR to n-day VaR

In what we have done so far, we used the 1-day VaR, but not everyone does. As we discussed above, among the criticism of VaR is that in periods of downturn markets, it might be difficult to liquidate positions, in particular when dealing in sizable in illiquid assets. If it were to take several days to liquidate positions, a10-day VaR could make more sense and provide a more realistic picture of the risk a financial institution faces. Here, there are two main approaches. For the sake of keeping things simple, we compare a1-day VaR to a30-day VaR. To obtain the 30-day VaR, we can follow two approaches:

We can take 30-day returns instead of 1-day returns. This is very simple and does not cause complications. However, we would be able to extract only 12 data points per year. That may not be enough to achieve the desired statistical precision in parametric approaches and definitely not enough to obtain reliable estimates using non-parametric methods, in particular on the tails of the distribution where precision requires a very large number of data points.

Alternatively, we could use a rolling average, i.e., we are considering periods of 30-days, but the periods are overlapping. This approach increases the number of data points with which we can work. However, now the 30-day returns are not uncorrelated as they use much of the same entries.

It is common practice to compute $1-$day VaR and then scale it up to n-day using scaling factors: The mean used in computing the $30 - day$ VaR is taken from that of the 1-day VaR and multiplied by 30; the standard deviation for the 30-day VaR is obtained by multiplying the volatility of the 1-day VaR by $\sqrt{30}$. However, this is correct only in those cases when prices follow a geometric Brownian motion and the returns a Brownian motion since in that case the properties of the normal distribution would allow us to do so.

7. Further Readings

An excellent paper that you should read at this point is "On Exactitude in Financial Regulation: Value-at-Risk, Expected Shortfall, and Expectiles." This paper summarizes several of the concepts we have seen in the lesson and expands on new ones, including the concept of *expectiles.* This is not a required reading, but it's highly encouraged.

8. Conclusion

In this lesson, we learned how to use VaR to assess portfolio risk and the severity of potential losses within given levels of confidence. This is very important. However, we need to also learn how to create good portfolios so that we can avoid risk as much as possible without giving up on investing in risky assets which could be necessary to generate certain target returns. This will be the scope of the next lesson.